

LeetCode 1293: Shortest Path in a Grid with Obstacles Elimination

© Authored by Arpit Raj, LNMIIT Jaipur

Date: July 23, 2026

Context & Previous Problems

1091, 1926, and 934 all had a state of:

(r, c)

→

$(r, c, kLeft)$

Now the state becomes $(r, c, kLeft)$ because reaching the same cell with different remaining eliminations is not the same state.

Example Scenario

For example:

Aadya reaches $(3, 4)$ with $k = 2$ and later reaches $(3, 4)$ with $k = 0$.

The first situation is much better because Aadya can still destroy two obstacles later.

"i juss hope that obstacle is not my heart TwT"

★ Asked in Goldman Sachs and PhonePe

Visualizing the 3D State (Added Context)

Why add $kLeft$ to the BFS State?

Standard BFS uses a 2D visited array seen $[r][c]$ to avoid infinite cycles. However, in this problem, we might need to revisit a cell if we discover a new path that is longer but

preserves more "obstacle elimination" power. The 3D array `seen[r][c][rem]` effectively layers multiple parallel dimensions over our grid, letting us systematically track the shortest path across different levels of remaining power.

Algorithm Implementation

```
class Solution {
public:
    vector<pair<int, int>> dir = {{-1, 0}, {0, -1}, {1, 0}, {0, 1}};

    int shortestPath(vector<vector<int>>& grid, int k) {

        int n = grid.size();
        int m = grid[0].size();
        vector<vector<vector<int>>> seen(n, vector<vector<int>>(m, vector<int>(k + 1, 0)));
        queue<vector<int>> q;

        // {row, col, remaining eliminations}
        q.push({0, 0, k});
        seen[0][0][k] = 1;

        int ans = 0;

        while (!q.empty()) {

            int currsz = q.size();

            while (currsz--) {

                int r = q.front()[0];
                int c = q.front()[1];
                int rem = q.front()[2];

                q.pop();

                if (r == n-1 && c == m-1) {
                    return ans;
                }

                for (auto it : dir) {
```

```

        int nr = r + it.first;
        int nc = c + it.second;

        if (nr >= 0 && nc >= 0 && nr < n && nc < m) {

            if (grid[nr][nc] == 0) { //matlab waha jaa skte without even u

                if (!seen[nr][nc][rem]) {

                    seen[nr][nc][rem] = 1;
                    q.push({nr, nc, rem});
                }
            }

            else { //matlab waha obstacle he

                if (rem > 0 && !seen[nr][nc][rem - 1]) { //obstacle tod ke

                    seen[nr][nc][rem - 1] = 1;
                    q.push({nr, nc, rem - 1});
                }
            }
        }
    }

    ans++;
}

return -1;
}
};

```